

DarshanEase – Temple Darshan Ticket Booking Web Application

Project Documentation

1. Introduction

Project Title:

DarshanEase – Temple Darshan Ticket Booking Web Application

Team Members:

Name	Role
Mohammad Tanzeela	Frontend Development
Alajingi Naga Pratyusha	Backend Development
Sravanthi Rachuri	Database Administrator
Neelam Prabhu Pragna Pauline	Authentication & API Developer
Thota Nagaraju	Testing and Documentation

2. Project Overview

Purpose:

DarshanEase is a full-stack MERN-based web application developed to provide a convenient and efficient platform for booking temple darshan tickets online. The application allows devotees to register, securely log in, browse available temples, select suitable darshan slots, book tickets, and view their booking details through an interactive user interface.

The main objective of DarshanEase is to simplify the traditional manual temple ticket booking process by providing a digital platform where users can access temple information and manage their darshan bookings easily.

The system is developed using modern MERN stack technologies including React.js, Node.js, Express.js, and MongoDB. It provides secure authentication,

efficient data management, and smooth communication between frontend and backend through RESTful APIs.

The application helps reduce waiting time, improves accessibility, and provides a better experience for devotees by enabling online darshan slot management.

Features:

The DarshanEase application provides the following major features:

- User registration and login system.
- Secure authentication using JWT (JSON Web Token).
- Password encryption using bcryptjs.
- Browse available temples.
- View temple details.
- Select and book available darshan slots.
- Manage user bookings.
- View previously booked darshan tickets.
- RESTful API communication between frontend and backend.
- MongoDB database integration for storing application data.
- Responsive and user-friendly interface.
- Secure user data management.

3. Architecture

DarshanEase follows a three-layer architecture consisting of:

1. Frontend Architecture
2. Backend Architecture
3. Database Architecture

This architecture improves scalability, maintainability, and efficient communication between different modules.

3.1 Frontend Architecture

The frontend of the DarshanEase application is developed using **React.js with Vite**. It provides an interactive and responsive interface for users to access different functionalities of the application.

React Router is used for navigation between different pages, while Axios is used for communication with backend REST APIs.

The frontend handles user interactions such as:

- User registration
- User login
- Temple browsing
- Slot selection
- Ticket booking
- Viewing booking history
- Managing user information

The frontend communicates with the backend server through HTTP requests and displays the received data dynamically.

Frontend Components:

The major frontend components include:

- Home Page
- User Registration Page
- Login Page
- Temple Listing Page
- Temple Details Page
- Darshan Slot Booking Page
- Booking History Page
- User Profile Page
- Dashboard

3.2 Backend Architecture

The backend of DarshanEase is developed using **Node.js and Express.js**.

The backend manages application logic, handles client requests, validates user data, communicates with MongoDB, and provides RESTful API services.

The backend provides services for:

- User authentication
- User management

- Temple information management
- Darshan booking operations
- Database communication

Middleware is used for authentication, authorization, and request validation.

Backend Components:

The backend consists of:

- Express Server
- API Routes
- Controllers
- Middleware
- Authentication Module
- Database Models
- Configuration Files

3.3 Database Architecture

DarshanEase uses **MongoDB**, a NoSQL database, for storing and managing application data.

Mongoose is used as the Object Data Modeling (ODM) library to define database schemas and interact with MongoDB.

The database stores information related to:

- Users
- Temples
- Darshan Slots
- Bookings
- Authentication Data

MongoDB provides flexible data storage and efficient retrieval of application records.

Main Collections:

Users Collection

Stores user information such as:

- Name
- Email
- Password
- User details

Temples Collection

Stores temple-related information:

- Temple name
- Location
- Description
- Available slots

Bookings Collection

Stores booking information:

- User details
- Temple details
- Selected darshan slot
- Booking status

3.4 Architecture Summary

DarshanEase follows a layered MERN architecture where React handles the user interface, Express and Node.js manage application logic, and MongoDB stores application data.

The frontend communicates with the backend using REST APIs. The backend interacts with MongoDB using Mongoose for database operations.

JWT authentication ensures secure access to protected resources, while bcryptjs provides password encryption.

This architecture makes the application secure, scalable, maintainable, and suitable for future enhancements.

4. Setup Instructions

Prerequisites:

Before running the DarshanEase application, ensure that the following software is installed on the system:

- **Node.js** (Version 16 or above)
- **Node Package Manager (npm)**
- **MongoDB Database**
- **Visual Studio Code**
- **Git**
- **Postman** (for API testing)
- **Google Chrome or any modern web browser**

These tools are required to install project dependencies, manage the database, run the application, and test API communication between frontend and backend.

Installation:

Follow the below steps to install and run the DarshanEase project:

Step 1: Clone Repository

Download or clone the project repository to your local system.

```
git clone https://github.com/sravanthirachuri28-gif/DarshanEase.git
```

Step 2: Open Project

Open the project folder in Visual Studio Code.

The project contains two major parts:

- Client (Frontend)
- Server (Backend)

Step 3: Install Frontend Dependencies

Navigate to the client folder:

```
cd client
```

Install required packages:

```
npm install
```

Start the frontend application:

```
npm run dev
```

Step 4: Install Backend Dependencies

Navigate to the server folder:

```
cd server
```

Install backend dependencies:

```
npm install
```

Start the backend server:

```
npm start
```

Step 5: Configure Database

Ensure that MongoDB is running locally or connect using MongoDB Atlas.

Create a configuration file containing:

- MongoDB connection URL
- Server port
- JWT secret key

Example:

```
MONGO_URI = your_mongodb_connection_string
```

```
JWT_SECRET = your_secret_key
```

```
PORT = 5000
```

After completing these steps, the DarshanEase application will be ready to use.

5. Folder Structure

The DarshanEase project follows a modular folder structure that separates frontend and backend components.

This separation improves code organization, readability, maintenance, and future scalability.

5.1 Client

The client side is responsible for providing the user interface and handling user interactions.

It is developed using React.js with Vite.

The frontend manages:

- User interface components
- Navigation
- API communication
- User input handling
- Data presentation

The client folder contains:

Components/

Contains reusable React components used throughout the application.

Examples:

- Navbar
- Forms
- Cards
- Booking components

Pages/

Contains different application pages.

Examples:

- Home Page
- Login Page
- Registration Page
- Temple Page

- Booking Page
- Profile Page

Assets/

Stores static resources such as:

- Images
- Icons
- Logos
- Other media files

Routes/

Handles navigation between different pages using React Router.

Services/

Contains API communication logic using Axios.

5.2 Server

The server side is developed using Node.js and Express.js.

It handles:

- Business logic
- Authentication
- API requests
- Database operations

The server contains the following folders:

config/

Stores configuration files such as:

- Database connection settings
- Environment configurations

controllers/

Contains functions that process API requests.

Controllers handle operations such as:

- User registration
- User login
- Temple data retrieval
- Booking operations

models/

Defines MongoDB schemas using Mongoose.

Main models include:

- User Model
- Temple Model
- Booking Model

routes/

Contains API route definitions.

Routes manage requests related to:

- Authentication
- Users
- Temples
- Bookings

middleware/

Contains middleware functions for:

- JWT verification
- Authentication checking
- Error handling

server.js

The main entry point of the backend application.

It:

- Creates Express server
- Connects MongoDB
- Registers routes
- Starts the application

package.json

Contains:

- Project information
- Dependencies
- Scripts required to execute the application

The project uses dependencies including:

- bcryptjs
- jsonwebtoken
- mongoose

6. Running the Application

6.1 Running the Frontend

Follow these steps:

1. Open the client folder in Visual Studio Code.
2. Open the terminal inside the client directory.
3. Install dependencies:

`npm install`

4. Start the React application:

`npm run dev`

5. The frontend application will start successfully.

The application can be accessed using the Vite development URL:

<http://localhost:5173>

6.2 Running the Backend

Follow these steps:

1. Open the server folder.
2. Open terminal in the server directory.
3. Install dependencies:

`npm install`

4. Ensure MongoDB is running.
5. Start the backend server:

`npm start`

or

`npm run dev`

6. The backend server will start and listen for API requests on the configured port.

Example:

`http://localhost:5000`

7. API Documentation

The DarshanEase application uses RESTful APIs for communication between the frontend and backend.

These APIs handle user authentication, temple management, and darshan booking operations.

The APIs exchange data in JSON format.

Authentication APIs

1. User Registration

Method:

POST

Endpoint:

/api/auth/register

Purpose:

Creates a new user account by storing user details securely in MongoDB.

2. User Login

Method:

POST

Endpoint:

/api/auth/login

Purpose:

Authenticates registered users and generates a JWT token.

User APIs

Get User Profile

Method:

GET

Endpoint:

/api/users/profile

Purpose:

Retrieves logged-in user information.

Update User Profile

Method:

PUT

Endpoint:

/api/users/profile

Purpose:

Updates user profile details.

Temple APIs

Get Temples

Method:

GET

Endpoint:

/api/temples

Purpose:

Retrieves available temple information.

Get Temple Details

Method:

GET

Endpoint:

/api/temples/:id

Purpose:

Retrieves details of a specific temple.

Booking APIs

The booking APIs handle darshan ticket reservations and booking management operations.

1. Create Darshan Booking

Method:

POST

Endpoint:

/api/bookings

Purpose:

Creates a new darshan ticket booking by storing user booking details in the database.

Request Data:

```
{  
  "templeId": "12345",  
  "date": "2026-07-20",  
  "slot": "10:00 AM",  
  "numberOfPersons": 2  
}
```

Response:

```
{  
  "message": "Booking created successfully",  
  "bookingId": "67890"  
}
```

2. Get User Bookings

Method:

GET

Endpoint:

/api/bookings

Purpose:

Retrieves all darshan bookings associated with the logged-in user.

3. Get Booking Details**Method:**

GET

Endpoint:

/api/bookings/:id

Purpose:

Retrieves complete information about a specific booking.

4. Cancel Booking**Method:**

DELETE

Endpoint:

/api/bookings/:id

Purpose:

Cancels or removes an existing darshan booking.

8. Authentication

The DarshanEase application implements secure authentication using **JSON Web Token (JWT)** and **bcryptjs password encryption**.

These security mechanisms ensure that user information remains protected and only authorized users can access restricted features.

User Registration

New users can create an account by entering required details such as:

- Name
- Email
- Password

During registration:

1. User information is validated.
2. Password is encrypted using bcryptjs.
3. Encrypted password is stored in MongoDB.
4. User account is created successfully.

User Login

Registered users can log into the system using their email and password.

During login:

1. User credentials are verified.
2. Password is compared using bcrypt validation.
3. A JWT token is generated after successful authentication.
4. Token is used for accessing protected routes.

JWT Authentication

JSON Web Token provides secure communication between frontend and backend.

The authentication process works as follows:

1. User logs in successfully.
2. Server generates a JWT token.
3. Token is sent to the client.
4. Client sends the token with protected API requests.
5. Server verifies the token before processing requests.

Authorization

Protected routes are secured using authentication middleware.

Only authenticated users can:

- Access their profile
- Book darshan slots
- View booking history
- Manage personal information

Security Features

The application provides:

- Password encryption using bcryptjs.
- JWT-based authentication.
- Protected API routes.
- Secure user data storage.
- Validation of user requests.

9. User Interfaces

The DarshanEase application provides a simple, responsive, and user-friendly interface that allows devotees to access all features easily.

The major screens of the application are:

9.1 Home Page

The Home Page acts as the landing page of the application.

It provides:

- Application introduction
- Navigation options
- Temple booking information
- User access options

9.2 Registration Page

The Registration Page allows new users to create an account.

Users enter:

- Name
- Email
- Password

The entered information is securely stored after password encryption.

9.3 Login Page

The Login Page allows existing users to securely access their accounts.

Features:

- Email verification
- Password authentication
- JWT token generation

9.4 Temple Listing Page

This page displays available temples with relevant details.

Users can view:

- Temple name
- Location
- Description
- Available darshan options

9.5 Temple Details Page

This page provides complete information about a selected temple.

Information includes:

- Temple description
- Darshan timings
- Available booking slots

9.6 Darshan Booking Page

The booking page allows users to reserve their darshan tickets.

Users can select:

- Temple
- Date
- Time slot
- Number of visitors

After confirmation, booking details are stored in the database.

9.7 Booking History Page

This page allows users to view previous bookings.

It displays:

- Temple name
- Booking date
- Darshan slot
- Booking status

9.8 User Profile Page

The profile page allows users to view and manage their personal information.

Features:

- View profile details
- Update user information
- Manage account details

10. Testing

The DarshanEase application was tested to ensure that all modules work correctly and efficiently.

The following testing methods were performed:

Functional Testing

Functional testing was performed to verify major application features:

- User registration
- User login
- Temple browsing
- Slot booking
- Booking history
- Profile management

All major functions were tested successfully.

API Testing

REST APIs were tested using tools such as Postman.

Testing included:

- Request validation
- Response verification
- Authentication verification
- Error handling

Database Testing

MongoDB operations were tested to ensure:

- Correct data storage
- Data retrieval
- Updating records
- Deleting records

Mongoose models were verified for proper database communication.

Authentication Testing

Authentication testing was performed to verify:

- JWT token generation
- Protected route access

- Password encryption
- Invalid login handling

Testing Outcome

All major functionalities of the DarshanEase application were tested successfully.

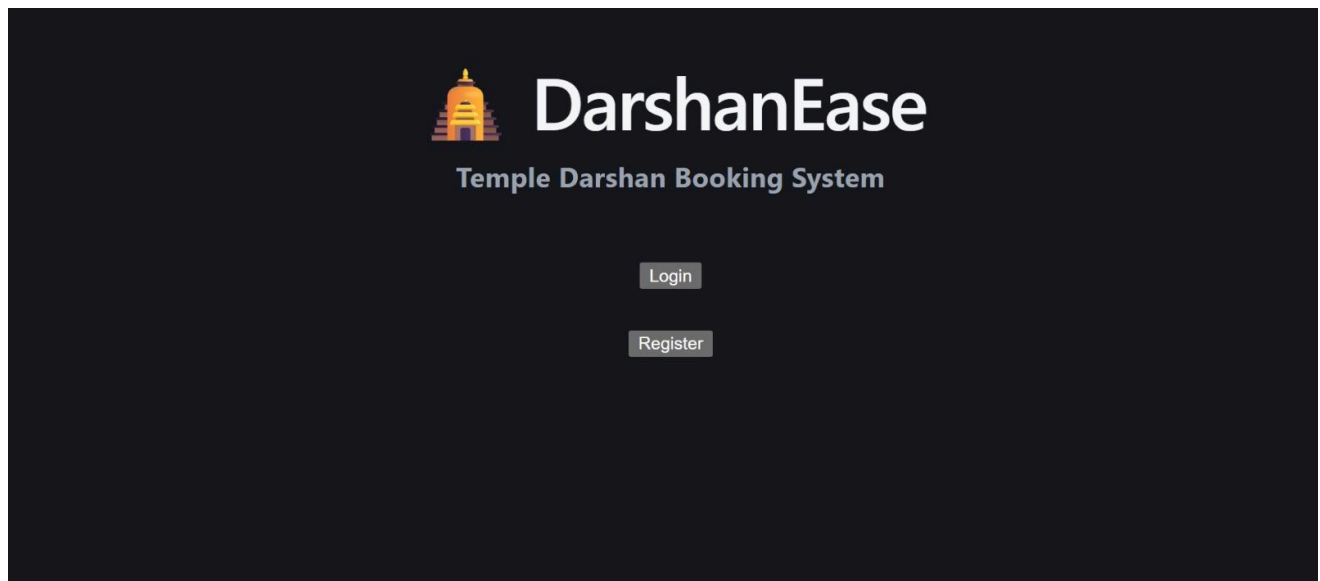
The application provides:

- Secure authentication
- Reliable database operations
- Smooth user experience
- Efficient booking management

11. Screenshots / Demo

The DarshanEase application consists of multiple user interface pages that provide a smooth and interactive experience for devotees. The screenshots below can be replaced with actual screenshots of your application during the final submission.

Home Page:



Registration Page:

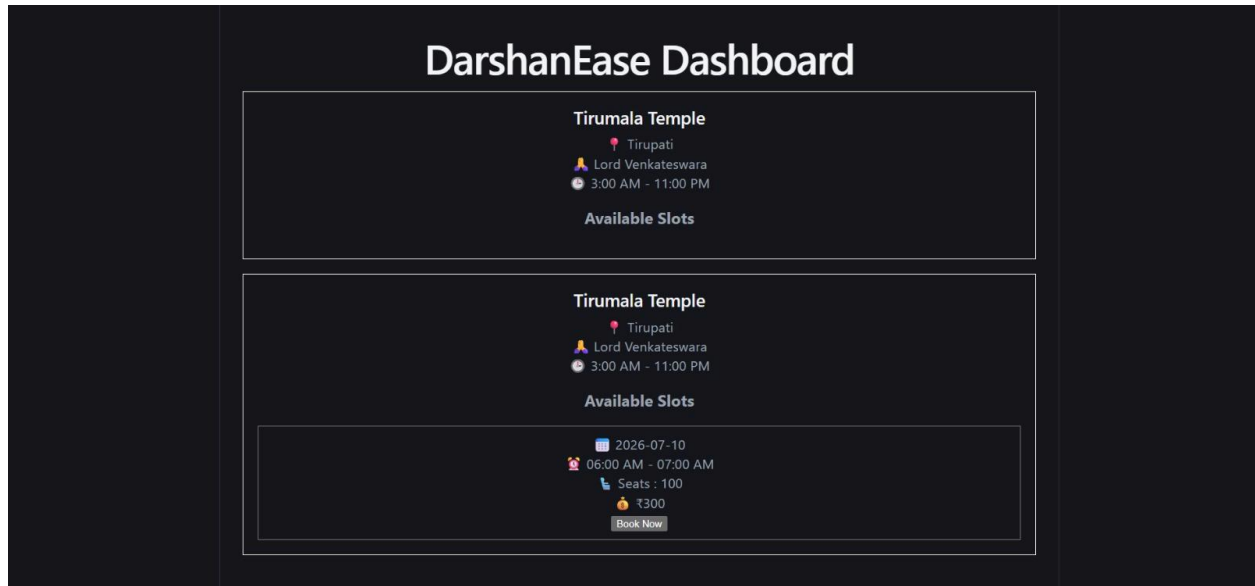
Register

Login Page:

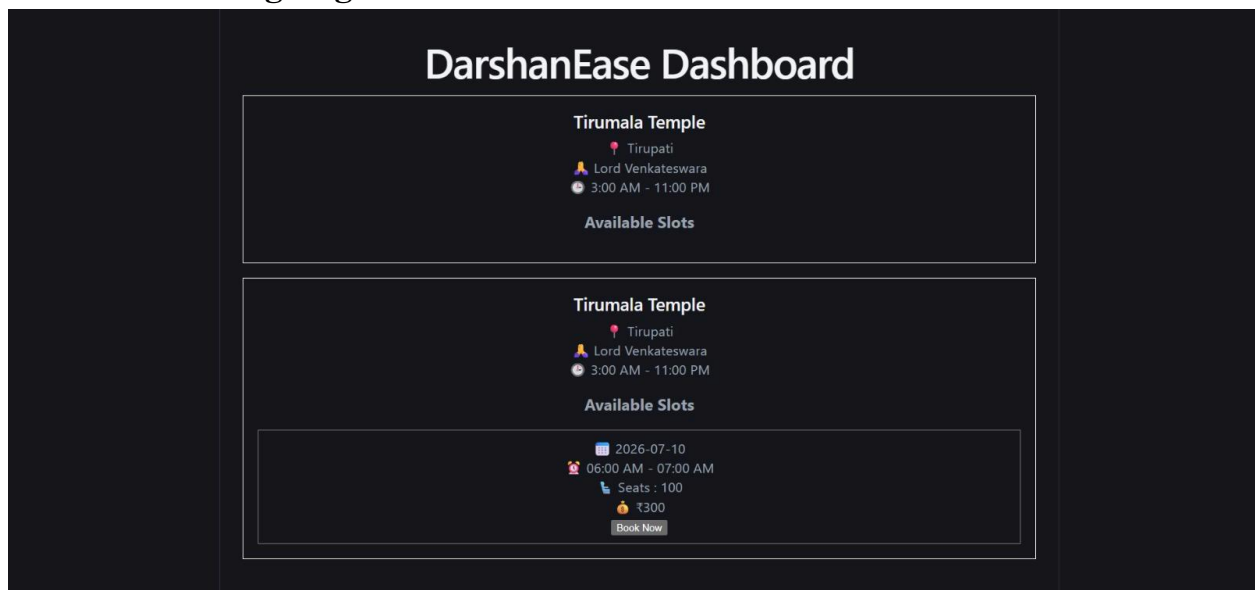
Login

Don't have an account? [Register](#)

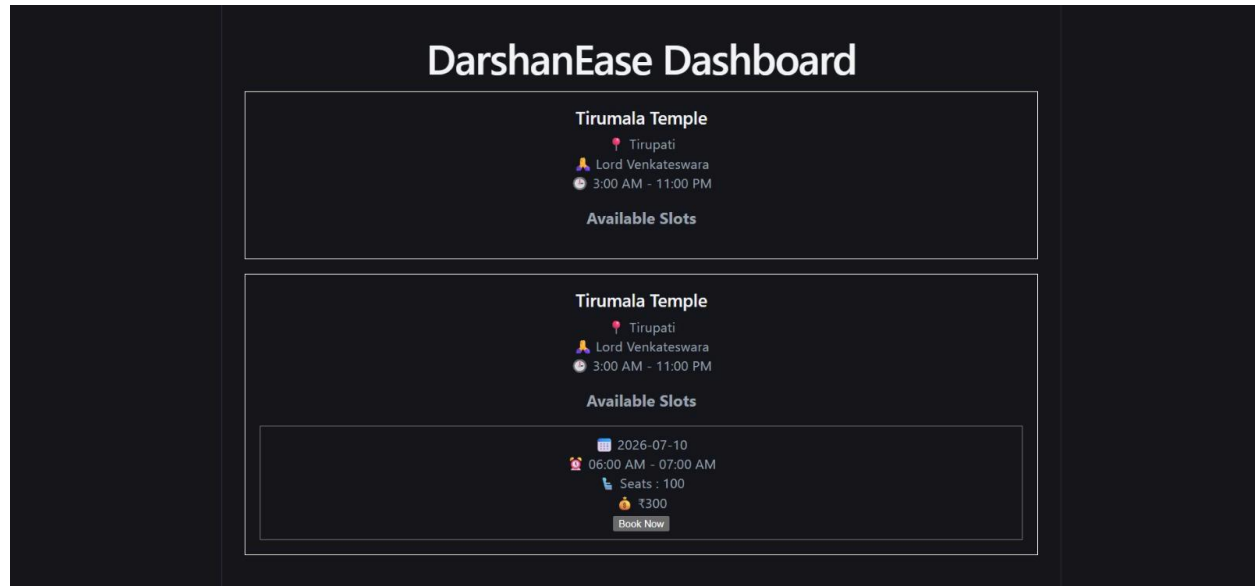
Temple Details Page:



Darshan Booking Page:



Booking History Page:



12. Known Issues

Although the DarshanEase application performs efficiently, a few limitations currently exist:

- A stable internet connection is required to access the application.
- MongoDB must be running locally or connected through MongoDB Atlas for proper database operations.
- Online payment integration has not yet been implemented.
- Email or SMS notifications for booking confirmation are currently unavailable.
- The application is designed primarily for web browsers and does not include a dedicated Android or iOS mobile application.
- During high traffic, simultaneous booking requests may require additional optimization to improve performance.

These limitations can be addressed in future versions of the application.

13. Future Enhancements

The DarshanEase application can be enhanced by implementing the following features:

- **Online Payment Gateway:** Integrate secure payment options such as Razorpay, Stripe, or UPI to enable online ticket payments.
- **QR Code-Based Darshan Tickets:** Generate QR codes for booked tickets to facilitate faster entry verification at temples.
- **Email and SMS Notifications:** Send booking confirmations, reminders, and cancellation updates to users.
- **Admin Dashboard:** Develop a dedicated admin panel to manage temples, bookings, users, and darshan slots.
- **Live Slot Availability:** Display real-time updates on available darshan slots to prevent overbooking.
- **Temple Search and Filters:** Allow users to search temples by name, city, state, or category.
- **Digital Donation Facility:** Enable devotees to make online donations directly through the application.
- **Multi-language Support:** Provide interfaces in multiple regional languages to improve accessibility for users across India.
- **Mobile Application:** Develop Android and iOS versions of DarshanEase for greater convenience.
- **Analytics Dashboard:** Implement reporting and analytics features to help administrators monitor bookings, user activity, and temple popularity.

Conclusion

The **DarshanEase – Temple Darshan Ticket Booking Web Application** successfully demonstrates the implementation of a secure, scalable, and user-friendly web application using the **MERN Stack**. The project simplifies the traditional temple darshan booking process by enabling devotees to register, browse temples, reserve darshan slots, and manage their bookings through an intuitive online platform.

By integrating **React.js** for the frontend, **Node.js** and **Express.js** for the backend, **MongoDB** for data storage, and **JWT with bcryptjs** for secure authentication, the application ensures reliable performance and strong security. The modular architecture also makes the system easy to maintain and extend.

Overall, DarshanEase achieves its primary objective of providing a convenient digital solution for temple darshan ticket booking. With future enhancements such as online payments, QR code tickets, notifications, and a mobile application, it has the potential to evolve into a comprehensive platform suitable for real-world deployment.